⚕ HERMES AGENT MASTER PLAN

Project: Looking Glass — Visual Memory System

Target IDE: VS Code | **Deploy:** GitHub Pages PWA | **License:** MIT

> **What is this document?** A complete, step-by-step execution blueprint for an autonomous Hermes Agent to scaffold, build, and iterate the Looking Glass project — a local-first, open-source Spatial alternative forked from `twitter-bookmarks-grid`. Every phase includes exact tools, terminal commands, file structures, and decision trees. No ambiguity. Read once, execute completely.

AGENT IDENTITY & OPERATING RULES

```
AGENT NAME:      Hermes
ROLE:            Full-Stack Engineering Agent + UX Strategist
WORKING DIR:     ~/projects/looking-glass/
NODE VERSION:    20.x LTS
PACKAGE MGR:     pnpm (preferred) or npm
COMMIT STYLE:    Conventional Commits (feat:, fix:, chore:, docs:)
BRANCH MODEL:    main → develop → feature/vX.X-*
```

Core Agent Constraints

1. **Never break working state.** Every commit must leave the app in a runnable condition.
2. **Test in Chrome + Safari (iOS sim) after every phase.** PWA behavior differs between engines.
3. **Prefer progressive enhancement.** Vanilla → React migration must be additive, not destructive.
4. **IndexedDB is the source of truth.** JSON file is always an export/import of that.
5. **Performance budget:** First meaningful paint < 1.5s. Canvas pan/zoom must hit 60fps.
6. **Accessibility floor:** Keyboard navigation + screen reader labels on all interactive

elements.

PHASE 0 — ENVIRONMENT BOOTSTRAP

Duration: ~30 min | Goal: Dev environment ready

0.1 — Required VS Code Extensions (Install Before Anything Else)

```
AGENT ACTION: Run these in VS Code Quick Open (Ctrl+P / Cmd+P)

ext install dbaeumer.vscode-eslint
ext install esbenp.prettier-vscode
ext install bradlc.vscode-tailwindcss
ext install ms-vscode.live-server
ext install GitHub.copilot
ext install eamodio.gitlens
ext install christian-kohler.path-intellisense
ext install PKief.material-icon-theme
ext install antfu.vite
ext install lokalise.i18n-ally
ext install wayou.vscode-todo-highlight
ext install usernamehw.errorlens
```

0.2 — VS Code Workspace Settings

Create `.vscode/settings.json`:

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.tabSize": 2,
  "editor.rulers": [100],
  "files.exclude": {
    "node_modules": true,
    "dist": false
  },
  "todo-highlight.keywords": [
    { "text": "AGENT:", "color": "#FF6B6B" },
    { "text": "HERMES:", "color": "#4ECDC4" },
    { "text": "FIXME:", "color": "#FFE66D" }
  ],
  "liveServer.settings.port": 5500,
  "liveServer.settings.root": "/",
```

```
  "typescript.preferences.importModuleSpecifier": "relative"
}
```

0.3 — System Tools Required

```
# Verify / install prerequisites
node --version          # Must be 20.x+
pnpm --version          # Install: npm i -g pnpm
git --version           # Must be 2.40+
gh --version            # GitHub CLI: brew install gh

# Optional but recommended
brew install jq          # For JSON wrangling in scripts
brew install watchman    # For fast file watching in large repos
```

0.4 — Fork & Clone

```
# Step 1: Fork via GitHub CLI
gh repo fork destefanis/twitter-bookmarks-grid --clone --remote

# Step 2: Move into project
cd twitter-bookmarks-grid

# Step 3: Rename project locally (don't rename repo yet — keep upstream tracking)
# We'll brand it Looking Glass in code, rename repo at V1.0 stable

# Step 4: Create develop branch
git checkout -b develop
git push -u origin develop

# Step 5: Tag the original state
git tag v0.0.0-upstream
git push origin --tags
```

PHASE 1 — V0.1: MINIMUM USABLE PRODUCT

Duration: 3-5 days | Branch: `feature/v0.1-infinite-canvas`

1.1 — Repo Restructure

AGENT ACTION: Reorganize directory layout BEFORE writing any new code.

```
looking-glass/
├── src/
│   ├── canvas/               # Canvas engine (pan/zoom/drag)
│   │   ├── CanvasEngine.js
│   │   ├── DragManager.js
│   │   └── ZoomManager.js
│   ├── cards/                # Card renderers by type
│   │   ├── BookmarkCard.js
│   │   ├── WebClipCard.js
│   │   └── BaseCard.js
│   ├── data/                 # Data layer
│   │   ├── store.js          # IndexedDB wrapper
│   │   ├── schema.js         # Item schema definitions
│   │   └── exporters.js      # JSON export/import
│   ├── ui/                   # UI chrome (sidebar, toolbar, modals)
│   │   ├── Sidebar.js
│   │   ├── Toolbar.js
│   │   └── Lightbox.js
│   ├── utils/
│   │   ├── meta-fetcher.js   # URL → OG metadata
│   │   └── helpers.js
│   ├── styles/
│   │   ├── tokens.css        # Design tokens
│   │   ├── canvas.css
│   │   └── cards.css
│   └── main.js               # Entry point
├── public/
│   ├── manifest.json         # PWA manifest
│   ├── sw.js                 # Service worker
│   └── icons/                # PWA icons (192, 512)
├── data/
│   └── bookmarks.json        # User bookmark data (gitignored)
├── scripts/
│   ├── generate-icons.js     # Auto-generate PWA icon set
│   └── validate-schema.js    # JSON schema validator
├── tests/
│   ├── canvas.test.js
│   └── store.test.js
├── .github/
│   └── workflows/
│       └── deploy.yml        # GitHub Pages deploy action
├── index.html
├── vite.config.js            # Added at V0.2 when we add build step
├── package.json
└── README.md
```

1.2 — Data Schema (Lock This In Early)

File: `src/data/schema.js`

```
/**
 * LOOKING GLASS — CANONICAL ITEM SCHEMA v0.1
 * All canvas items conform to this shape.
 * Extend via `meta` and `style` — never break base fields.
 */

export const ITEM_TYPES = {
  BOOKMARK:  'bookmark',    // X/Twitter bookmark
  WEB_CLIP:  'web_clip',   // URL paste → OG card
  NOTE:      'note',        // Sticky note (V0.2)
  IMAGE:     'image',       // Uploaded image (V0.2)
  VIDEO:     'video',       // Video embed (V0.3)
  PDF:       'pdf',         // PDF viewer (V0.4)
  GROUP:     'group',       // Item container/stack (V0.2)
};

export const createItem = (overrides = {}) => ({
  // Identity
  id:          crypto.randomUUID(),
  type:        ITEM_TYPES.WEB_CLIP,
  created_at:  Date.now(),
  updated_at:  Date.now(),

  // Canvas position & size
  x:           0,
  y:           0,
  width:       320,
  height:      null,          // null = auto height
  rotation:    0,             // degrees (V0.2)
  z_index:     0,

  // Content
  content: {
    title:       '',
    description: '',
    url:         null,
    image_url:   null,
    text:        null,        // For notes
    file_path:   null,        // For local files
    embed_html:  null,        // For rich embeds
  },

  // Source metadata
```

```
  meta: {
    source:      'manual',    // 'twitter', 'url_paste', 'upload', 'extension'
    tags:        [],
    color:       null,        // User-set card accent
    pinned:      false,
    archived:    false,
    group_id:    null,        // Parent group item ID
    twitter_id:  null,        // Original tweet ID if applicable
    domain:      null,        // Parsed from URL
    read_at:     null,
    fetch_status: 'pending',  // 'pending' | 'ok' | 'failed' | 'manual'
  },

  // Visual overrides
  style: {
    background: null,
    text_color: null,
    font_size:  null,
    opacity:    1,
  },
});

export const CANVAS_STATE_SCHEMA = {
  version:    '0.1.0',
  id:         '',             // Canvas UUID
  name:       'My Canvas',
  created_at: Date.now(),
  updated_at: Date.now(),
  viewport: {
    x:      0,
    y:      0,
    scale:  1,
  },
  items:      [],             // Array of Item objects
};
```

1.3 — IndexedDB Store

File: `src/data/store.js`

```
/**
 * AGENT INSTRUCTIONS:
 * Implement a clean IndexedDB wrapper using idb-keyval for simplicity,
 * with fallback to localStorage for Safari private mode.
 *
 * Methods to implement:
```

```
 * - store.init()                    → Open/upgrade DB
 * - store.getCanvas(id)             → Return full canvas state
 * - store.saveCanvas(state)         → Upsert canvas
 * - store.listCanvases()            → Return all canvas summaries
 * - store.getItem(id)               → Single item
 * - store.upsertItem(item)          → Create or update
 * - store.deleteItem(id)            → Soft delete (archived flag)
 * - store.hardDeleteItem(id)        → Permanent removal
 * - store.bulkImport(items)         → Batch import from JSON
 * - store.exportCanvas(id)          → Returns full JSON
 *
 * DB NAME:    'looking-glass-db'
 * VERSION:    1
 * STORES:
 *   - canvases:  keyPath='id'
 *   - items:     keyPath='id', indexes=['canvas_id', 'type', 'meta.tags']
 *   - blobs:     keyPath='id'  (for file storage)
 *
 * USE idb package: pnpm add idb
 */
```

1.4 — Canvas Engine (Core Pan/Zoom)

File: `src/canvas/CanvasEngine.js`

```
/**
 * AGENT INSTRUCTIONS:
 * Build a performant infinite canvas using CSS transforms (NOT canvas element).
 * This approach allows native DOM for cards (accessibility, text selection).
 *
 * Architecture:
 * - #canvas-viewport  (overflow: hidden, captures pointer events)
 *   └── #canvas-world (transform: translate(x,y) scale(s), transform-origin: 0 0
 *       └── .canvas-item (each card, absolutely positioned)
 *
 * Pan: Pointer events on viewport. On pointerdown (not on card), track delta.
 *      Apply to world transform. Use requestAnimationFrame for smooth updates.
 *
 * Zoom: wheel event → adjust scale. Zoom toward cursor position (requires
 *       transform math: new translate = cursor - (cursor - old_translate) * (new
 *       Clamp scale: MIN=0.1, MAX=3.0
 *
 * Pinch: Touch events. Track two-finger distance delta for scale.
 *        Midpoint of fingers = zoom origin.
 *
 * Performance:
```

```
 * - Use will-change: transform on canvas-world
 * - Batch DOM reads/writes via requestAnimationFrame
 * - Virtual culling: Only render items within expanded viewport rect (add 200px
 *   Check visibility: item.x + item.width > viewLeft && item.x < viewRight (adju
 *
 * State: { x, y, scale } — persist to canvas.viewport in store on pointerup/whee
 *
 * Methods:
 * - init(containerEl)
 * - panTo(x, y, animate=false)
 * - zoomTo(scale, originX, originY)
 * - fitToContent()           → calculate bounds of all items, zoom to fit
 * - getViewportBounds()      → { left, top, right, bottom } in world coords
 * - worldToScreen(x, y)      → converts world coords to screen px
 * - screenToWorld(x, y)      → inverse transform
 */
```

1.5 — Drag Manager

File: **`src/canvas/DragManager.js`**

```
/**
 * AGENT INSTRUCTIONS:
 * Handle dragging of individual canvas items.
 *
 * Requirements:
 * - Items draggable by their header/handle area (not entire card)
 * - Multi-select + drag group: Shift+click to multi-select, drag moves all
 * - Snap-to-grid: Optional. Toggle via Toolbar. Grid size: 20px.
 * - Snap-to-edge: Show alignment guides when within 8px of another item's edge
 * - Touch support: Single finger on card header = drag (not pan)
 * - During drag: apply grabbing cursor, slight scale(1.02) + shadow for lift eff
 * - On drop: save new {x, y} to store (debounced 500ms)
 * - Z-index management: Bring dragged item to front during drag
 *
 * Alignment Guides:
 * - Render as SVG overlay lines on the canvas-world element
 * - Show: left-align, right-align, center-align, top-align, bottom-align
 * - Auto-hide 300ms after drag ends
 *
 * Boundary:
 * - Items cannot be dragged to negative infinity — soft boundary at x=-5000, y=-
 * - No right/bottom boundary (infinite canvas)
 */
```

1.6 — URL Metadata Fetcher

File: `src/utils/meta-fetcher.js`

```
/**
 * AGENT INSTRUCTIONS:
 * Fetch Open Graph + Twitter Card metadata from a URL.
 *
 * Strategy (try in order, fall back on failure):
 *
 * 1. PRIMARY — allorigins.win proxy:
 *    https://api.allorigins.win/get?url=ENCODED_URL
 *    Parse response HTML for OG tags
 *
 * 2. FALLBACK A — microlink.io API (free tier):
 *    https://api.microlink.io/?url=ENCODED_URL
 *    Returns JSON with title, description, image, etc.
 *
 * 3. FALLBACK B — manual entry:
 *    Return partial object { url, fetch_status: 'failed' }
 *    UI shows "couldn't fetch, enter details manually"
 *
 * Tags to extract:
 *   og:title, og:description, og:image, og:image:width, og:image:height
 *   twitter:card, twitter:title, twitter:image
 *   <title>, <meta name="description">
 *   <link rel="icon"> (for favicon)
 *   canonical URL
 *
 * Return shape:
 * {
 *   title: string,
 *   description: string,
 *   image_url: string | null,
 *   favicon_url: string | null,
 *   domain: string,
 *   canonical_url: string,
 *   fetch_status: 'ok' | 'failed' | 'partial',
 * }
 *
 * Cache results in sessionStorage keyed by URL hash to avoid re-fetching.
 * Timeout: 8 seconds per attempt before trying next fallback.
 */
```

1.7 — PWA Setup

File: `public/manifest.json`

```
{
  "name": "Looking Glass",
  "short_name": "Looking Glass",
  "description": "Your visual memory — bookmarks, web clips, ideas on an infinite
  "start_url": "/",
  "display": "standalone",
  "background_color": "#0a0a0f",
  "theme_color": "#0a0a0f",
  "orientation": "any",
  "categories": ["productivity", "utilities"],
  "icons": [
    { "src": "/icons/icon-192.png", "sizes": "192x192", "type": "image/png", "pur
    { "src": "/icons/icon-512.png", "sizes": "512x512", "type": "image/png", "pur
    { "src": "/icons/icon-180.png", "sizes": "180x180", "type": "image/png" }
  ],
  "screenshots": [],
  "share_target": {
    "action": "/share",
    "method": "POST",
    "enctype": "multipart/form-data",
    "params": {
      "title": "title",
      "text": "text",
      "url": "url"
    }
  }
}
```

File: `public/sw.js` — Agent must implement:

- Cache-first strategy for app shell (HTML, CSS, JS)
- Network-first for metadata fetch calls
- Background sync queue for failed store writes
- Push notification registration (stubbed for future use)

1.8 — Design System & Tokens

File: `src/styles/tokens.css`

```
/* LOOKING GLASS DESIGN TOKENS
   Aesthetic: Dark Brutalist / Analog-Digital
```

```
   Inspired by: Braun product design meets early-web directory aesthetics
   NOT: generic purple SaaS gradients
*/

:root {
  /* ── Color ──────────────────────────────── */
  --color-bg:           #0a0a0f;    /* Near black with blue undertone */
  --color-bg-elevated:  #111118;    /* Card surfaces */
  --color-bg-overlay:   #16161f;    /* Modals, tooltips */

  --color-border:       #1e1e2e;
  --color-border-active:#3a3a5c;

  --color-accent:       #c8fb4a;    /* Electric chartreuse — the signature color
  --color-accent-dim:   rgba(200, 251, 74, 0.12);
  --color-accent-hover: #d9ff5a;

  --color-text-primary: #e8e8f0;
  --color-text-secondary:#7a7a9a;
  --color-text-muted:   #3d3d5a;
  --color-text-inverse: #0a0a0f;

  /* Semantic card accents (user-selectable) */
  --card-red:           #ff6b6b;
  --card-amber:         #ffb347;
  --card-green:         #69f0ae;
  --card-blue:          #64b5f6;
  --card-purple:        #ce93d8;
  --card-none:          var(--color-border);

  /* ── Typography ─────────────────────────── */
  --font-display:       'Fragment Mono', 'Courier New', monospace;
  --font-body:          'DM Sans', system-ui, sans-serif;
  --font-ui:            'DM Mono', 'Courier New', monospace;

  --text-xs:   11px;
  --text-sm:   13px;
  --text-base: 15px;
  --text-lg:   18px;
  --text-xl:   22px;
  --text-2xl:  28px;

  --leading-tight:  1.2;
  --leading-normal: 1.5;

  /* ── Spacing ────────────────────────────── */
  --space-1:  4px;
```

```
  --space-2:  8px;
  --space-3:  12px;
  --space-4:  16px;
  --space-5:  20px;
  --space-6:  24px;
  --space-8:  32px;
  --space-10: 40px;
  --space-12: 48px;

  /* ── Radius ──────────────────────────── */
  --radius-sm:  4px;
  --radius-md:  8px;
  --radius-lg:  12px;
  --radius-xl:  20px;
  --radius-pill:9999px;

  /* ── Shadow ──────────────────────────── */
  --shadow-card:   0 2px 8px rgba(0,0,0,0.4), 0 0 0 1px var(--color-border);
  --shadow-lifted: 0 12px 40px rgba(0,0,0,0.6), 0 0 0 1px var(--color-border-acti
  --shadow-accent: 0 0 20px rgba(200, 251, 74, 0.15);

  /* ── Motion ──────────────────────────── */
  --ease-spring:  cubic-bezier(0.34, 1.56, 0.64, 1);
  --ease-smooth:  cubic-bezier(0.4, 0, 0.2, 1);
  --duration-fast:   120ms;
  --duration-normal: 220ms;
  --duration-slow:   400ms;

  /* ── Canvas ──────────────────────────── */
  --canvas-grid-color: rgba(255,255,255,0.04);
  --canvas-grid-size:  40px;
  --card-min-width:    240px;
  --card-max-width:    480px;
  --card-width-default:300px;
}
```

1.9 — GitHub Pages Deploy Workflow

File: `.github/workflows/deploy.yml`

```
name: Deploy Looking Glass to GitHub Pages

on:
  push:
    branches: [main]
  workflow_dispatch:
```

```
permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: false

jobs:
  deploy:
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'pnpm'
      - run: pnpm install --frozen-lockfile
      - run: pnpm build
      - uses: actions/configure-pages@v4
      - uses: actions/upload-pages-artifact@v3
        with:
          path: ./dist
      - uses: actions/deploy-pages@v4
        id: deployment
```

1.10 — V0.1 Acceptance Criteria (Agent Must Verify All)

```
[ ] Canvas pans smoothly on desktop (mouse drag on empty space)
[ ] Canvas pans on iOS Safari (single finger on empty space)
[ ] Canvas zooms with scroll wheel (desktop)
[ ] Canvas zooms with pinch gesture (iOS)
[ ] 50 bookmark cards load without jank
[ ] Cards drag and drop to new positions
[ ] Positions persist after page refresh (IndexedDB)
[ ] Pasting a URL into the URL bar creates a card with OG metadata
[ ] PWA installs to iOS home screen without errors
[ ] JSON export downloads valid schema-conforming file
[ ] JSON import loads and renders cards correctly
[ ] Filters (color, tag, type) narrow visible cards
[ ] Lightbox opens on card click, closes on overlay click / Esc
```

```
[ ] Lighthouse PWA score > 80
```

PHASE 2 — V0.2: CONTENT TYPES & ORGANIZATION

Duration: 1-2 weeks | Branch: `feature/v0.2-content-types`

2.1 — Migrate to Vite Build System

```
pnpm add -D vite @vitejs/plugin-legacy
```

`vite.config.js`：

```
import { defineConfig } from 'vite'
import legacy from '@vitejs/plugin-legacy'

export default defineConfig({
  base: '/looking-glass/',  // GitHub Pages path
  build: {
    outDir: 'dist',
    sourcemap: true,
    rollupOptions: {
      output: {
        manualChunks: {
          canvas: ['./src/canvas/CanvasEngine.js'],
        }
      }
    }
  },
  plugins: [
    legacy({ targets: ['iOS >= 14', 'Chrome >= 90'] })
  ]
})
```

2.2 — New Card Types to Implement

Card Type	Description	Key UI Elements
`note`	Sticky note with markdown	Edit on double-click, auto-resize to content
`image`	Drag-drop / paste image upload	Shows image, resize handle

`video`	YouTube/Vimeo embed	Thumbnail → play inline on click
`web_clip`	Full URL card (enhanced)	OG image, favicon, title, description, domain badge

2.3 — Stack/Group System

```
/**
 * AGENT INSTRUCTIONS — GROUP SYSTEM:
 *
 * A Group is an item of type 'group' that owns child items.
 * Children reference parent via meta.group_id.
 *
 * Group rendering:
 * - Collapsed: Single card showing child count + preview thumbnails
 * - Expanded: Children rendered with slight offset (stacked fan) or in a boundin
 * - Header bar: Group name + collapse/expand button + item count badge
 *
 * Creation:
 * - Select 2+ items → "Group" action in context menu or toolbar
 * - Auto-calculate bounding box from children positions
 * - Move group → moves all children maintaining relative positions
 *
 * Dissolve: "Ungroup" action → items remain in place, group item deleted
 *
 * Auto-group suggestions (from bookmark tags/filters):
 * - Group by 'tag' → cluster by tag
 * - Group by 'domain' → cluster by website domain
 * - Group by 'type' → bookmarks vs web clips vs notes
 * Show as "Suggested Groups" in sidebar, one-click to apply
 */
```

2.4 — Search Implementation

```
/**
 * AGENT INSTRUCTIONS — SEARCH:
 *
 * Use Fuse.js for fuzzy search: pnpm add fuse.js
 *
 * Search index fields (weighted):
 *   content.title:       weight 3
 *   content.description: weight 2
 *   content.text:        weight 2
 *   meta.tags:           weight 2
```

```
 *   content.url:         weight 1
 *   meta.domain:         weight 1
 *
 * UX:
 * - Command+K / Ctrl+K opens search overlay (full-screen, centered input)
 * - Results appear as mini-card list below input (max 12 visible, scroll)
 * - Clicking result: pan+zoom canvas to that item, flash highlight ring
 * - ESC closes
 * - Filter chips below input: type filter, tag filter, date range
 * - "Jump to" vs "Add to selection" button on each result
 */
```

2.5 — Undo/Redo System

```
/**
 * AGENT INSTRUCTIONS — HISTORY MANAGER:
 *
 * Implement a command pattern with a history stack.
 * Max stack depth: 100 operations
 *
 * Commands to implement:
 * - MoveItemCommand(id, fromPos, toPos)
 * - CreateItemCommand(item)
 * - DeleteItemCommand(item)
 * - ResizeItemCommand(id, fromSize, toSize)
 * - GroupCommand(items, group)
 * - UngroupCommand(group, items)
 * - UpdateContentCommand(id, from, to)  ← for notes
 *
 * Keyboard: Cmd+Z / Ctrl+Z = undo, Cmd+Shift+Z / Ctrl+Y = redo
 * Show undo/redo available state in toolbar buttons (disabled when empty stack)
 * Toast notification: "Moved 3 items" / "Undone" for user awareness
 */
```

2.6 — V0.2 New Dependencies

```
pnpm add fuse.js              # Fuzzy search
pnpm add idb                  # IndexedDB wrapper
pnpm add markdown-it          # Markdown rendering for notes
pnpm add mime                 # MIME type detection for file uploads
pnpm add -D vite @vitejs/plugin-legacy
```

PHASE 3 — V0.3: POLISH & WORKFLOW

Duration: 1-2 weeks | Branch: `feature/v0.3-polish`

3.1 — Multiple Canvases ("Spaces")

```
/**
 * AGENT INSTRUCTIONS — SPACES SYSTEM:
 *
 * A Space = one complete canvas with its own items, viewport state, name.
 * Stored as separate records in canvases store.
 *
 * UI: Spaces switcher in left sidebar — like browser tabs but vertical.
 * Default space: "Main" (created on first launch)
 *
 * Space operations:
 * - Create new space (+ button)
 * - Rename space (double click label)
 * - Duplicate space (right-click context menu)
 * - Delete space (must have 1+ space remaining)
 * - Reorder spaces (drag in sidebar)
 *
 * Cross-space item move: Drag item to space tab → moves to that space at center
 *
 * Space card: Optional "Space Card" = a card on canvas that is a portal/link
 *             to another space. Click to navigate. Thumbnail preview on hover.
 */
```

3.2 — Export System

Format	Method	Notes
JSON	Full canvas state	Lossless, re-importable
PNG	html2canvas library	Full canvas at current zoom, with option for 2x
PDF	jsPDF + html2canvas	Paginated grid of cards
Markdown	Custom serializer	Exports notes, titles, URLs as `.md` file
Obsidian	Markdown + frontmatter	Compatible with Obsidian Canvas format

```
pnpm add html2canvas jspdf     # Export
```

```
pnpm add jszip                  # Bundle exports into .zip
```

3.3 — iOS Bottom Sheet UI

```
/**
 * AGENT INSTRUCTIONS — MOBILE UI OVERHAUL:
 *
 * On screens < 768px, switch to mobile-optimized layout:
 *
 * Bottom Sheet Toolbar:
 * - Dock at bottom (safe area inset aware)
 * - Left: Spaces selector (icon + name truncated)
 * - Center: Core actions (add URL, add note, search)
 * - Right: Canvas controls (zoom fit, minimap toggle)
 *
 * Context Menu on mobile:
 * - Long press card → haptic feedback → bottom sheet slides up
 * - Options: Edit, Move to Space, Group, Color, Delete
 * - NO right-click context menu on mobile (pointer: coarse)
 *
 * Card Sizing on Mobile:
 * - Default card width: min(90vw, 320px)
 * - Initial layout: 1-column stack at x=20, y=increasing
 *
 * iOS PWA Fixes:
 * - Handle viewport-fit=cover in CSS (env(safe-area-inset-*))
 * - Disable double-tap zoom: touch-action: manipulation on canvas
 * - Status bar: dark-content (for dark app background)
 * - Splash screen meta tags for all iOS device sizes
 */
```

3.4 — Minimap Component

```
/**
 * AGENT INSTRUCTIONS — MINIMAP:
 *
 * Small overview map in bottom-right corner.
 * Canvas element, not DOM (performance).
 *
 * Renders:
 * - Each item as a colored rectangle (color by type or user color)
 * - Current viewport as a white rectangle overlay
 *
 * Interaction:
```

```
 * - Click on minimap → pan canvas to that location
 * - Drag viewport rect on minimap → live pan
 * - Toggle button to show/hide
 *
 * Update: Throttle to 30fps (no need for 60fps)
 * Scale: Minimap covers all item bounds + 20% padding
 * Size: 160×100px on desktop, 120×80px on mobile (or hidden)
 */
```

PHASE 4 — V0.4–V1.0: FULL SPATIAL FEATURES

Duration: 3-6 weeks | Branch: `feature/v0.4-spatial-parity`

4.1 — React Migration Strategy

AGENT ACTION: Gradual migration — wrap vanilla modules as React components.

```
pnpm add react react-dom
pnpm add -D @vitejs/plugin-react
pnpm add @tanstack/react-query    # Data fetching/cache
pnpm add zustand                  # State management (lighter than Redux)
pnpm add framer-motion            # Animation (replaces CSS transitions at scale)
pnpm add @radix-ui/react-*        # Accessible UI primitives
pnpm add cmdk                     # Command palette (better than custom search)
pnpm add react-hotkeys-hook       # Keyboard shortcuts
```

Migration Order:

1. UI chrome first (Sidebar, Toolbar, Modals) — low risk
2. Card renderers — medium risk
3. Canvas engine last — highest risk, most testing needed

4.2 — SQLite Integration (Advanced Data Layer)

```
/**
 * AGENT INSTRUCTIONS — SQLITE VIA ABSURD-SQL:
 * For power users with 10k+ items, IndexedDB becomes slow.
 * Add SQLite as optional enhanced storage backend.
 *
 * Package: pnpm add @sqlite.org/sqlite-wasm
 * Or: pnpm add absurd-sql (simpler API)
```

```
 *
 * Tables:
 *   CREATE TABLE items (
 *     id TEXT PRIMARY KEY,
 *     canvas_id TEXT NOT NULL,
 *     type TEXT NOT NULL,
 *     x REAL, y REAL, width REAL, height REAL, rotation REAL, z_index INTEGER,
 *     content TEXT,  -- JSON blob
 *     meta TEXT,     -- JSON blob
 *     style TEXT,    -- JSON blob
 *     created_at INTEGER, updated_at INTEGER,
 *     FOREIGN KEY(canvas_id) REFERENCES canvases(id)
 *   );
 *   CREATE INDEX idx_items_canvas ON items(canvas_id);
 *   CREATE INDEX idx_items_type   ON items(type);
 *   CREATE VIRTUAL TABLE items_fts USING fts5(id, title, description, text);
 *
 * Feature flag: User can switch between IndexedDB and SQLite in Settings.
 * Both use same store.js API — swap underlying implementation.
 */
```

4.3 — Full-Text Search (SQLite FTS5)

Replace Fuse.js with SQLite FTS5 for power users:

```
-- Indexing
INSERT INTO items_fts(id, title, description, text)
VALUES (?, ?, ?, ?);

-- Search
SELECT items.* FROM items
JOIN items_fts ON items.id = items_fts.id
WHERE items_fts MATCH ?
ORDER BY rank;
```

4.4 — Rich Markdown Editor

```
pnpm add @tiptap/react @tiptap/starter-kit @tiptap/extension-*
```

Editor features for Note cards:

- Slash commands (`/heading` , `/list` , `/table` , `/divider` , `/image`)
- Markdown shortcuts (`**bold**` , `## heading`)

- `@mention` for linking to other canvas items (bi-directional)
- Keyboard shortcut to exit edit mode (Esc)
- Auto-save debounced 1s after last keystroke

4.5 — PDF Support

```
pnpm add pdfjs-dist
```

```
/**
 * AGENT INSTRUCTIONS — PDF VIEWER CARD:
 * - Load PDF from URL or local file via pdfjs-dist
 * - Render first page as thumbnail on card
 * - Click → expand to full-screen PDF viewer overlay
 * - Features: page navigation, zoom, text selection
 * - Card shows: filename, page count, file size badge
 */
```

PHASE 5 — V2.0: GOD TIER FEATURES

Duration: Ongoing | Branch: `feature/v2.0-god-tier`

5.1 — AI Integration Layer

```
/**
 * AGENT INSTRUCTIONS — AI MODULE:
 *
 * Provide AI features via user-supplied API keys (stored in IndexedDB, never sen
 * Support: Anthropic Claude API + Google Gemini API + OpenAI API
 *
 * Key: Settings → AI → Paste API key. Key stored encrypted in IndexedDB.
 *
 * Feature: "Ask Canvas" — Command+J
 *   Open floating chat panel. User asks natural language questions.
 *   Context sent to AI: titles + descriptions of all visible items.
 *   Examples: "Find all UI inspiration", "Summarize this cluster", "What themes
 *
 * Feature: "Auto-Tag" button on card context menu
 *   Send card content to AI → returns suggested tags array → apply with user con
 *
 * Feature: "Generate Note" from selection
```

```
 *   Select items → "Synthesize" → AI generates a markdown note summarizing selec
 *   New Note card placed at center of selection
 *
 * Feature: "Find Connections"
 *   Embed all items (via AI embeddings API) → compute cosine similarity
 *   Draw SVG lines between highly similar items (similarity > 0.85)
 *   Line thickness = similarity strength
 *   Toggle: "Show AI Connections" in View menu
 *
 * Feature: "Smart Auto-Group"
 *   K-means clustering on embeddings → suggest groupings to user
 *   Preview as highlighted outlines before confirming
 *
 * Rate limiting: Queue AI calls, max 5 concurrent. Show queue status in toolbar.
 */
```

5.2 — Browser Extension (Looking Glass Clipper)

```
Extension Target: Chrome + Safari (via Xcode wrapper)
Tech: Manifest V3

Actions:
1. Click extension icon → "Clip this page" → sends {url, title, screenshot, selec
2. Right-click selection → "Save to Looking Glass" context menu
3. Keyboard shortcut: Cmd+Shift+L

Communication:
- With LG web app open: postMessage via content script
- With LG app closed: Store in extension storage, import on next open

Screenshot: Use chrome.tabs.captureVisibleTab for full-page screenshot
Metadata: Extract og:image, twitter:card, canonical URL from page DOM

Files:
  extension/
  ├── manifest.json
  ├── background.js
  ├── content.js
  ├── popup/
  │   ├── popup.html
  │   └── popup.js
  └── icons/
```

5.3 — Real-Time Collaboration (Optional Self-Hosted)

```
pnpm add yjs y-websocket          # CRDT-based sync
pnpm add @hocuspocus/server       # WebSocket server (self-hosted)
```

```
/**
 * AGENT INSTRUCTIONS — COLLABORATION:
 *
 * Use Yjs CRDT (Conflict-free Replicated Data Types) for multi-user.
 * Each item's {x, y, content} tracked as Yjs shared types.
 *
 * Modes:
 * 1. Local-only (default) — no collaboration
 * 2. P2P (no server) — WebRTC via y-webrtc, share code to invite
 * 3. Self-hosted (advanced) — user deploys hocuspocus server via Docker
 *
 * Presence indicators:
 * - Show colored cursor labels for other users (name + colored ring)
 * - Item locked indicator when another user is editing
 * - Live "typing" on Note cards (show other user's cursor in text)
 *
 * Docker Compose for self-hosted backend (provide in repo):
 *   services:
 *     hocuspocus:
 *       image: hocuspocus/hocuspocus:latest
 *       ports: ["1234:1234"]
 */
```

5.4 — Advanced Import System

Source	Method	Priority
Twitter/X bookmarks	fieldtheory-cli JSON	P0 (existing)
Raindrop.io	Raindrop API + CSV import	P1
Readwise	Readwise API (highlights)	P1
Notion	Notion API (pages → cards)	P1
Obsidian	Vault folder import (md files)	P2
Browser bookmarks	HTML bookmarks file	P2
Instapaper	CSV export	P2
Pocket	CSV export	P2

Are.na	Are.na API	P3
Pinterest	Pinterest API	P3

5.5 — Knowledge Graph View

```
/**
 * AGENT INSTRUCTIONS — GRAPH VIEW:
 *
 * Toggle: View menu → "Graph Mode"
 * Renders canvas items as force-directed graph (D3.js or Sigma.js)
 *
 * Nodes: Each item. Size by connection count. Color by type.
 * Edges:
 *   - Manual @mention links (solid line)
 *   - Same tag (dashed line, low opacity)
 *   - AI semantic similarity (gradient line, thickness = strength)
 *   - Same group (dotted)
 *
 * Interaction:
 *   - Drag nodes → updates x,y on canvas (sync back)
 *   - Click node → focus sidebar on that item
 *   - Double-click node → jump to item on main canvas
 *   - Filter: Show only connections of type X
 *
 * pnpm add d3 / pnpm add sigma graphology graphology-layout-forceatlas2
 */
```

AGENT TOOLCHAIN REFERENCE

Complete Package Registry

```
# ── Runtime Dependencies ──────────────────────────
pnpm add idb                          # IndexedDB wrapper
pnpm add fuse.js                      # Fuzzy search (V0.2)
pnpm add markdown-it                  # Markdown rendering
pnpm add mime                         # MIME type detection
pnpm add html2canvas                  # Canvas-to-PNG export
pnpm add jspdf                        # PDF export
pnpm add jszip                        # Zip bundling
pnpm add react react-dom              # React (V0.4+)
```

```
pnpm add zustand                       # State management
pnpm add framer-motion                 # Animations
pnpm add @tiptap/react @tiptap/starter-kit  # Rich text editor
pnpm add pdfjs-dist                    # PDF viewer
pnpm add fuse.js                       # Fuzzy search
pnpm add yjs y-webrtc y-websocket      # Real-time sync (V2.0)
pnpm add d3                            # Graph view
pnpm add @sqlite.org/sqlite-wasm       # SQLite (V0.4+)
pnpm add cmdk                          # Command palette
pnpm add react-hotkeys-hook            # Keyboard shortcuts
pnpm add @radix-ui/react-dialog        # Accessible modals
pnpm add @radix-ui/react-tooltip       # Tooltips
pnpm add @radix-ui/react-context-menu # Right-click menus
pnpm add @tanstack/react-query         # Data fetching
pnpm add @tanstack/react-virtual       # Virtual list rendering

# ── Dev Dependencies ──────────────────────────
pnpm add -D vite @vitejs/plugin-react @vitejs/plugin-legacy
pnpm add -D vitest @testing-library/react @testing-library/jest-dom
pnpm add -D playwright @playwright/test
pnpm add -D eslint prettier eslint-config-prettier
pnpm add -D typescript @types/react @types/react-dom
pnpm add -D tailwindcss postcss autoprefixer
pnpm add -D @storybook/react           # Component docs/dev (optional)
```

External APIs & Services (Free Tier)

Service	Use	Limit
`api.allorigins.win`	CORS proxy for OG metadata	1000 req/day
`api.microlink.io`	Structured metadata extraction	100 req/day
`fonts.googleapis.com`	DM Sans, DM Mono, Fragment Mono	Unlimited
`cdnjs.cloudflare.com`	CDN for dependencies	Unlimited

TESTING STRATEGY

Unit Tests (Vitest)

```
// tests/schema.test.js
```

```
import { createItem, ITEM_TYPES } from '../src/data/schema.js'

describe('Item Schema', () => {
  it('creates item with required fields', () => {
    const item = createItem({ type: ITEM_TYPES.NOTE })
    expect(item.id).toBeDefined()
    expect(item.type).toBe('note')
    expect(item.x).toBe(0)
  })
})
```

Key areas to unit test:

- Schema validation (`createItem` , defaults, overrides)
- Store CRUD operations (mock IndexedDB)
- Meta fetcher (mock fetch, all fallback paths)
- Canvas math (world↔screen transforms, zoom math)
- History manager (undo/redo stack)

E2E Tests (Playwright)

```
// tests/e2e/canvas.spec.js
test('drag and drop card updates position', async ({ page }) => {
  await page.goto('/')
  // Load fixture data
  // Drag a card 200px right
  // Verify store has updated x position
  // Reload page, verify position persists
})
```

Key E2E scenarios:

- Full bookmark import flow
- URL paste → card creation
- Drag card → reload → position persists
- Multi-canvas creation and switching
- JSON export/import round trip
- PWA install prompt (Chrome)

- iOS Safari touch pan/pinch

```
# Run tests
pnpm test            # Vitest unit tests
pnpm test:e2e        # Playwright
pnpm test:coverage   # Coverage report
```

PERFORMANCE BENCHMARKS

The agent must track and meet these targets at every milestone:

Metric	Target	Measure With
First Contentful Paint	< 1.2s	Lighthouse
Time to Interactive	< 2.0s	Lighthouse
Canvas pan (60 items)	60fps	DevTools Performance
Canvas pan (500 items)	55fps+	DevTools Performance
Canvas pan (1000 items)	45fps+	DevTools Performance
IndexedDB write (single)	< 10ms	console.time
Metadata fetch (URL)	< 3s	Network panel
Bundle size (initial)	< 150KB gzipped	Vite Bundle Analyzer
Lighthouse PWA	> 90	Lighthouse
Lighthouse Accessibility	> 85	Lighthouse

SECURITY CHECKLIST

```
[ ] API keys stored in IndexedDB, never in localStorage, never in URL params
[ ] Content Security Policy headers set (no inline scripts in production)
[ ] All fetched HTML parsed with DOMParser, never innerHTML'd without sanitizatio
[ ] User-supplied URLs validated before fetch (must be http/https)
[ ] File uploads: MIME type verified, size limited to 50MB per file
[ ] No eval() usage anywhere in codebase
```

```
[ ] Dependency audit: pnpm audit run before every release
[ ] HTTPS enforced (GitHub Pages provides this automatically)
[ ] Share Target API input sanitized before processing
```

PROGRESSIVE ROLLOUT MILESTONES

```
V0.1.0 — Infinite Canvas (Target: Week 1)
  Deployable, personally usable. 50+ bookmarks + web clips on canvas.

V0.1.5 — PWA Stable (Target: Week 2)
  iPhone home screen working. Sync via JSON export/import.

V0.2.0 — Content Types (Target: Week 3-4)
  Notes, images, groups, search. Post on X for first feedback.

V0.3.0 — Multi-Canvas + Export (Target: Week 5-6)
  Spaces, undo/redo, export formats. Share with 5-10 beta users.

V1.0.0 — Public Launch (Target: Week 7-10)
  React migration complete. Full spatial parity with get-spatial.com.
  ProductHunt launch. README polished with screenshots/demo GIF.

V1.5.0 — AI Layer (Target: Month 3-4)
  Claude/Gemini API integration. Auto-tag, ask canvas, connections.

V2.0.0 — Ecosystem (Target: Month 5-6)
  Browser extension. Collaboration. Plugin API. Second ProductHunt.
```

HERMES AGENT DECISION TREE

```
WHEN STARTING A NEW FEATURE:
  1. Read schema.js — does this feature require schema changes?
     YES → Update schema, write migration, bump DB version
     NO  → Continue
  2. Does this feature require new dependencies?
     YES → Check bundle impact (vite-bundle-analyzer), add if < 20KB gzipped
     NO  → Continue
  3. Is this a canvas-touching change?
     YES → Test on iOS Safari before committing
     NO  → Continue
  4. Write unit test first (TDD preferred)
```

```
  5. Implement feature
  6. Test manually: desktop Chrome → desktop Safari → iOS Safari
  7. Run: pnpm test && pnpm build (must pass clean)
  8. Commit with conventional commit message
  9. Update CHANGELOG.md

WHEN A PERFORMANCE ISSUE IS FOUND:
  1. Profile in Chrome DevTools Performance tab
  2. Identify whether bottleneck is: layout/paint, JS execution, or I/O
  3. Layout/Paint → Check will-change, reduce paint areas, virtualize off-screen
  4. JS execution → Debounce, move to Web Worker, reduce loop complexity
  5. I/O → Batch IndexedDB writes, add request queue

WHEN A BUG IS FOUND ON IOS:
  1. Check if it's a Safari-specific CSS issue (flexbox gap, transforms, etc.)
  2. Check Touch event vs Pointer event handling
  3. Check safe-area-inset CSS variables
  4. Check if PWA-specific (standalone mode behaves differently)
  5. Always test fix in actual iOS Safari — emulator is not sufficient
```

CHANGELOG TEMPLATE

File: `CHANGELOG.md` — Agent maintains this file after every milestone.

```
# Looking Glass Changelog

## [Unreleased]

## [0.1.0] - YYYY-MM-DD
### Added
- Infinite canvas with pan/zoom
- X bookmark import via fieldtheory JSON
- URL paste → OG card creation
- IndexedDB persistence
- PWA manifest + service worker
- GitHub Pages deployment

## [0.0.0] - YYYY-MM-DD
### Initial
- Fork of twitter-bookmarks-grid
```

README TEMPLATE

Agent must update `README.md` to this structure at V0.1.0:

```
# 🪞 Looking Glass

> Your visual memory. Bookmarks, web clips, ideas — on an infinite canvas.

[Live Demo](https://yourusername.github.io/looking-glass/) ·
[Add to iPhone](https://yourusername.github.io/looking-glass/) ·
[Roadmap](#roadmap)

![Screenshot](./docs/screenshot.png)

## Features (V0.1)
- 🗂️ Import X bookmarks via fieldtheory-cli
- 🌐 Paste any URL → fetch title, image, description automatically
- 🖱️ Infinite canvas — pan, zoom, drag cards freely
- 📱 iPhone-ready PWA — add to home screen
- 💾 Auto-saves to your browser (IndexedDB)
- 📤 Export / Import your canvas as JSON

## Quick Start
...

## Import Your X Bookmarks
...

## Roadmap
...

## Contributing
...

## License
MIT
```

End of Hermes Agent Master Plan v1.0 — Looking Glass Project Generated for autonomous agent execution in VS Code. Review all AGENT INSTRUCTIONS comments before beginning each phase.